



第一章 环境与简介 (上海-悠悠)

第一章 环境与简介 (上海-悠悠)

1.1 Selenium 发展史

- 1.1.1 Selenium 1.0
- 1.1.2 Selenium 2.0
- 1.1.3 Selenium 3.0
- 1.1.4 Selenium 4.0

1.2 后 Selenium 时代

- 1.2.1 Cypress 简介
- 1.2.2 Puppeteer
- 1.2.3 TestCafe
- 1.2.4 为什么要学 Playwright ?

1.3 playwright 环境准备

- 1.3.1 环境准备
- 1.3.2 运行代码快速体验

1.4 快速开始

- 1.4.1 简单使用
- 1.4.2 headless 模式
- 1.4.3 关于等待

1.5 录制生成脚本

- 1.5.1 启动运行
- 1.5.2 录制相关命令操作
- 1.5.3 page.pause() 断点调试

1.6 pause断点 inspector 使用

- 1.6.1 page.pause() 断点
- 1.6.2 console 调试定位
- 1.6.3 playwright inspector 使用

1.7 Playwright+Pytest+Allure POM项目

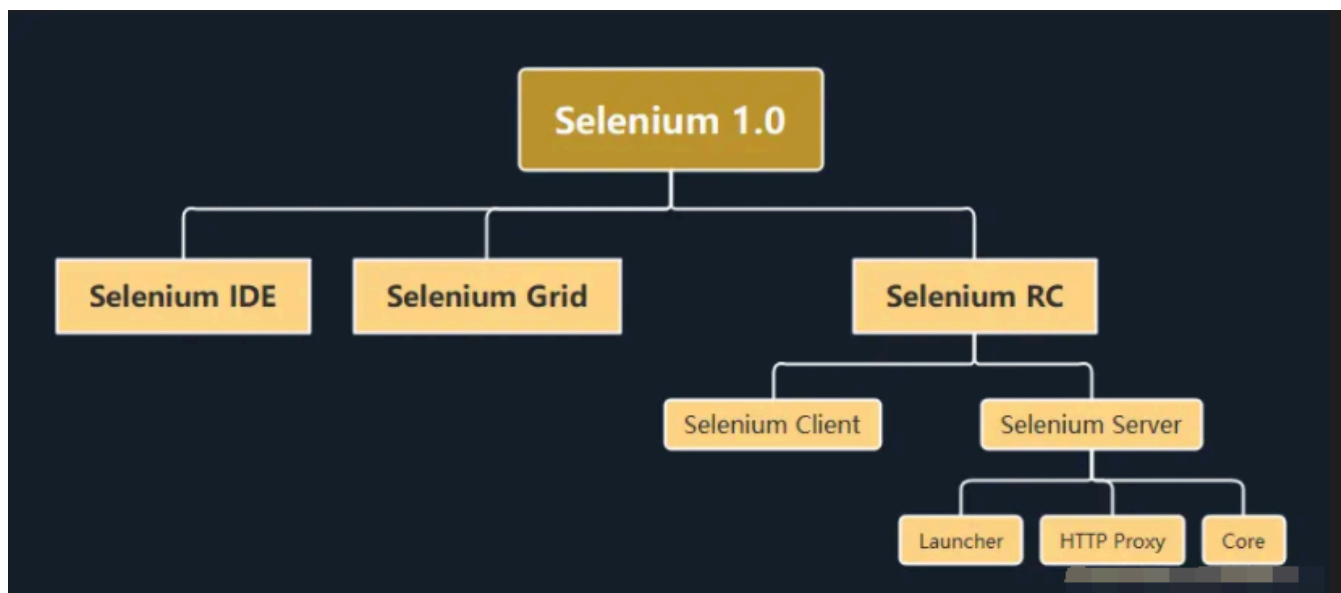
- 1.7.1 POM 设计模式
- 1.7.2 运行用例

- 网易云视频课程

1.1 Selenium 发展史

1.1.1 Selenium 1.0

在 Selenium 1.0 发布的 2006 年，Google 工程师 Simon Stewart 发起了一个名为 WebDriver 的项目。它也是一个自动化测试工具，彼时刚刚起步，后来它也将成为 Selenium 的竞品之一。

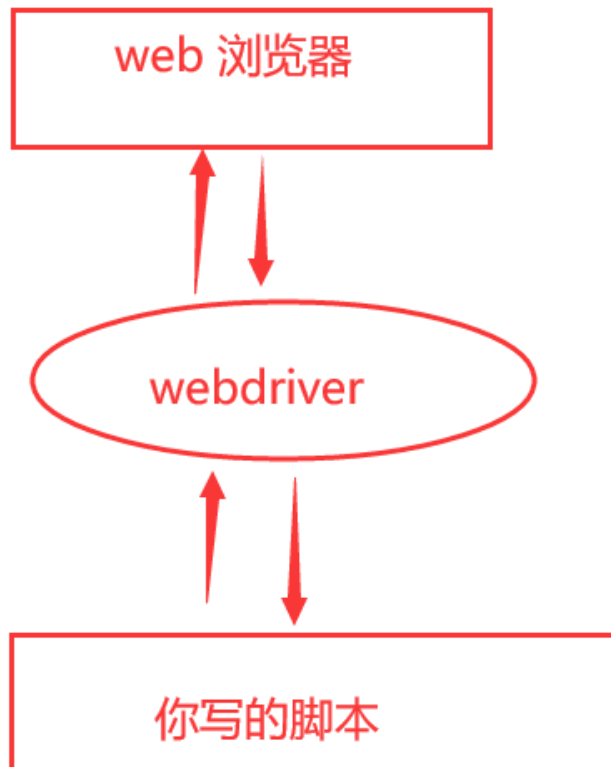


1.1.2 Selenium 2.0

2009 年，在 Google 测试自动化会议上，两个团队的开发人员在沟通后决定合并这两个项目，新项目被命名为 Selenium Web Driver，也就是 Selenium 2.0。

WebDriver 的作者是这样解释二者合并的原因的：“一方面 WebDriver 解决了 Selenium 存在的缺点（例如可以绕过 JavaScript 沙箱，WebDriver 有出色的 API），另一方面是 Selenium 解决了 WebDriver 存在的问题，还有就是 Selenium 的主要贡献者和我都以为合并项目是为用户提供最优秀框架的最佳途径。”

Selenium 2.x (WebDriver) 真正的兴起是在 2014 年开始，到 2016 年左右成为 web 自动化最热门的框架。几乎只要一说 web 自动化，那就是 selenium，它不仅仅在 web 自动化测试领域很火，在爬虫领域也是非常热门的，可以说全国不管哪行的都开始学 selenium 了。



Selenium Remote Control (RC) : 支持多种平台(Windows, Linux, Solaris)和多种浏览器(IE, Firefox, Opera, Safari), 可以用多种语言(Java, Ruby, Python, Perl, PHP, C#)

1.1.3 Selenium 3.0

2016 年, Selenium 3 发布。这个版本并没有引入新的工具, 主要加强了对浏览器的支持。

相较 Selenium 2 的主要的变动有 :

完全移除了 Selenium RC。

WebDriver 暴露一个供浏览器接入的 API, 通过各浏览器厂商提供的 Driver 来接入。

将 Firefox Driver 剔除 (之前 Firefox Driver 是内置的)。

支持 Firefox 通过 GECKO Driver 来接入 Selenium。

通过 Apple 提供的 Safari Driver, Selenium 可以支持 Safari 接入。

通过 Edge Driver 支持 IE 接入。

1.1.4 Selenium 4.0

2021 年, Selenium 发布 Selenium 4。

在 Selenium 3 中，与浏览器的通信基于 JSON-wire 协议，因此 Selenium 需要对 API 进行编解码。而 Selenium 4 遵循 W3C 标准协议，Driver 与浏览器之间通信的标准化使得他们可以直接通信。

除此之外，Selenium 4 还做了很多改动。包括：

优化了对浏览器的支持。

使用新的设计优化了 Selenium Grid。

标准化了 Selenium 的文档（你敢信从 Selenium 2.0 开始，文档就没更新过...）。

IDE 中的 CLI Runner 变更为基于 NodeJS（之前是 HTML Runner）。

Client 和 Driver 支持了新的元素定位 API。

支持屏幕截图。

改进了 Chrome Dev Tools。之前 Chrome Driver 直接继承自 Remote Web Driver 类，现在继承自 Chromium Driver 类，这个改动使得 IDE 开发可以使用更多的 API。

1.2 后 Selenium 时代

Selenium 的辉煌时代源于 webdriver 的引入，在2020年逐步走向没落，

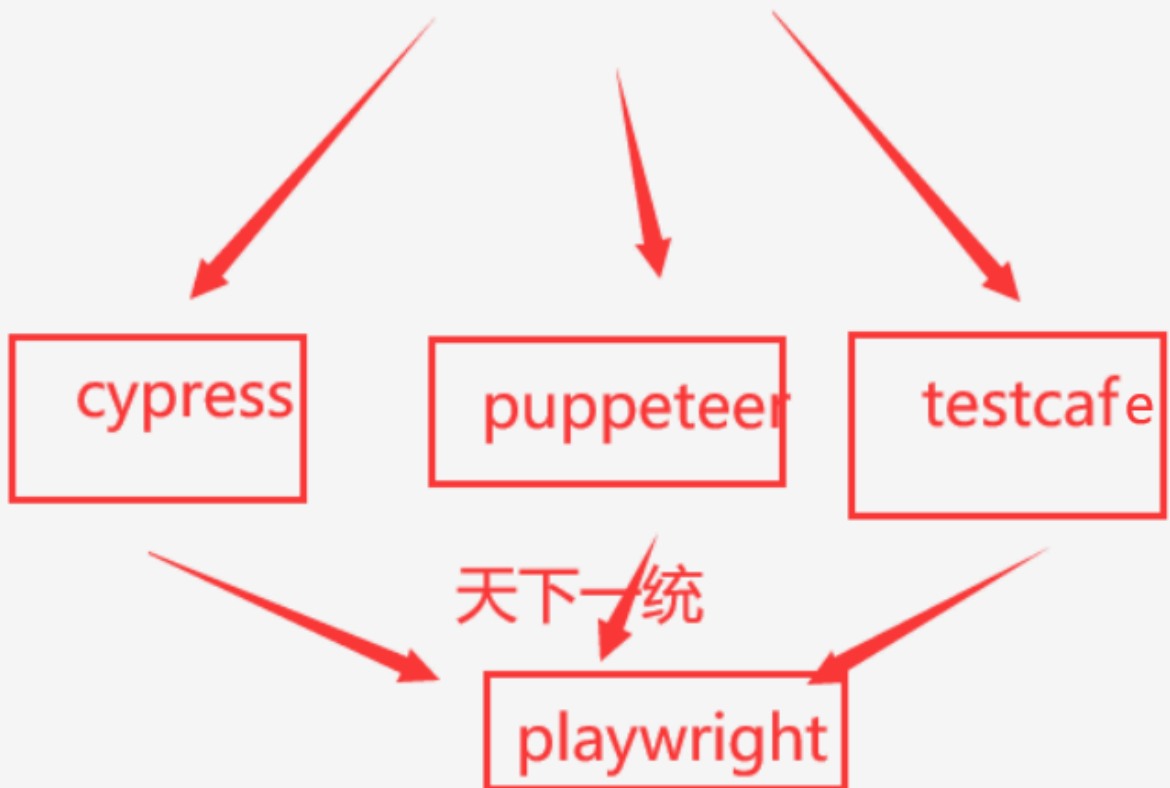
最终成也 webdriver 败也webdriver!

在2020年左右，Cypress、TestCafe、Puppeteer在技术雷达中被誉为后Selenium时代Web UI测试的三驾马车。

web 自动化

selenium 统一天下

末年，三分天下



1.2.1 Cypress 简介

Cypress 的第一版于2015年发布，_并于2018年开始真正流行，2020年,Cypress正式开源
Cypress官网地址：<https://docs.cypress.io>

随便着Web应用项目的不断发展，测试也要不断发展，Cypress是为现代网络打造的，基于JavaScript语言的一种前端自动化测试工具，测试也是如此，对浏览器中运行的所有内容进行快速，轻松和可靠的测试。Cypress是自集成的，这类测试框架统称为e2e测试，即end to

end（端到端）测试。理论上前端页面由前端框架来测试确实更为合适。它提供了一套完整的端到端的测试体验，正如官网上所说：

1. 安装Cypress很简单,无需依赖项,无需额外下载或更改代码。
2. 轻松，快速地编写测试，并在构建Web应用程序时观察它们的实时执行。
3. 在CI中调试测试就像在本地运行测试一样容易。具有内置的并行化和负载平衡功能。
4. 记录CI测试数据，屏幕截图和视频-并在“仪表盘”中查看汇总的下一级见解。

八大功能：

1. 时间穿梭：Cypress会在测试运行时拍摄快照。只需将鼠标悬停在“命令日志”中的命令上，即可准确查看每一步都发生了什么
2. 可调试性：停止猜测你的测试失败的原因。直接从熟悉的工具如Chrome DevTools进行调试。我们可读的错误和堆栈跟踪使调试更加快速便捷。
3. 实时重载：每当你对测试进行更改时，Cypress都会自动重新加载。查看应用程序中实时执行的命令。
4. 自动等待：不要在测试中添加等待或休眠。Cypress在继续下一步之前会自动等待元素至可操作状态时才会执行命令或断言。异步操作不再是噩梦。
5. 间谍，存根和时钟：Cypress允许你验证并控制函数行为，Mock服务器响应或更改系统时间，你喜欢的单元测试就在眼前。
6. 网络流量控制：Cypress可以Mock服务器返回结果，无须连接后端服务器即可实现轻松控制，模拟网络请求。
7. 运行结果一致性：Cypress架构不使用Selenium或Webdriver,在运行速度，可靠性测试，测试结果一致性上均有良好的保障。
8. 截图和视频：Cypress在测试运行失败时自动截图，在无头运行时录制整个测试套件的视频，使你轻松掌握测试运行情况。

1.2.2 Puppeteer

Puppeteer 是 Chrome 开发团队在 2017 年发布的一个 Node.js 包, puppeteer 是google公司出品 对于运行无界面chrome (学名headless)。

Puppeteer 是一个 Node 库，它提供了一个高级 API 来通过 DevTools 协议控制 Chromium 或 Chrome。

Puppeteer 默认以无头模式（headless）运行，也就是运行一个无界面的 Chrome 浏览器。

2 应用场景

2.1 页面生成 PDF

Puppeteer 提供了页面生成 PDF 的方法，我们可以利用这个方法将页面导出为 PDF，导出的 PDF 效果和 Chrome 浏览器打印功能导出的 PDF 一致。

具体的应用场景有：

- 报表导出 PDF
- 在线文档导出 PDF

2.2 页面截图

Puppeteer 提供了截图的方法，我们可以利用这个方法将页面的指定区域导出为 jpeg 或 png 图片。

具体的应用场景有：

- 报表导出图片
- 在线文档导出图片
- 营销页面生成预览图、封面图
- 生成骨架屏图片

2.3 服务端渲染

单页应用（SPA）的主要内容是在 JavaScript 向服务端请求数据后渲染的，存在爬虫难以抓取主要内容、首屏加载慢等问题，而使用 Next.js、Nuxt.js 等服务端渲染框架改造的成本较高。

如果只是为了搜索引擎优化，我们可以考虑利用 Puppeteer 来实现。我们可以在网关层判断请求的来源，如果是爬虫，直接返回由 Puppeteer 服务端渲染的 html 文件。

2.4 自动化UI测试

使用 Puppeteer 可以模拟 Chrome 浏览器环境，结合 JavaScript 测试框架（如 Jest）可以实现自动化 UI 测试。

Puppeteer 提供了 Mouse 类来模拟鼠标操作，提供了 Keyboard 类来模拟键盘操作，提供了 Touchscreen 类来模拟触屏操作，并且 Puppeteer 提供的 Page 类里有很多方法可以用来操作元素，比如点击元素、聚焦元素等操作

1.2.3 TestCafe

2018 年11月TestCafe首次进入技术雷达_,处于评估阶段,2019年4月的技术雷达里,它的状态更新为试验阶段

TestCafe 是一款 Node.js 库, 用来实现 web页面的自动化测试, 支持多种浏览器, 将来还有接口测试的计划。

下面通过和 **Puppeteer** 的对比来总结一些 TestCafe 的特点 (顺便带上 selenium) :

对比	Puppeteer	TestCafe	Selenium
语言	Javascript	Javascript/TypeScript	多种 binding 语言
兼容浏览器	仅Chrome系列	多种浏览器 (包括 IE)	多种浏览器 (包括 IE)
测试框架	不面向测试, 无框架	本身就是测试框架	无框架
工作原理	通过WebSocket 协议与 chrome 的通信	通过设置代理, 将用户的行为传递给被测页面	通过 wire protocol 和 浏览器 通信

1.2.4 为什么要学 Playwright ?

2020年微软开源一个 UI 自动化测试工具 Playwright, 支持 Node.js、Python、C# 和 Java 语言。

当 Microsoft 于2020 年 1 月 31 日发布Playwright的第一个公共版本时, 我们获得了一个新选项。

如果您比较Playwright和 Puppeteer的贡献者页面, 您会注意到Puppeteer的前两个贡献者现在在 Playwright 上工作。Puppeteer 团队实质上是从 Google 转移到 Microsoft 并成为 Playwright 团队。

因此, Playwright 在很多方面与 Puppeteer 非常相似。API 方法在大多数情况下是相同的, 并且默认情况下 Playwright 还捆绑了兼容的浏览器。

Playwright 最大的区别在于跨浏览器支持。它可以驱动 Chromium、WebKit (Safari 的浏览器引擎) 和 Firefox。

那么现在微软推出的 Playwright 到底有没必要去学呢? 先看下官方介绍<https://playwright.dev/python/>

跨浏览器和平台

- 跨浏览器。Playwright 支持所有现代渲染引擎，包括 Chromium、WebKit（Safari 的浏览器引擎）和 Firefox。
- 跨平台。在 Windows、Linux 和 macOS 上进行本地测试或在 CI 上进行无头或有头测试。
- 跨语言。在 TypeScript、JavaScript、Python、.NET、Java 中使用 Playwright API。
- 测试移动网络。适用于 Android 和 Mobile Safari 的 Google Chrome 浏览器的本机移动仿真。相同的渲染引擎适用于您的桌面和云端。

稳定性

- 自动等待。Playwright 在执行动作之前等待元素可操作。它还具有一组丰富的内省事件。两者的结合消除了人为超时的需要——这是不稳定测试的主要原因。
- Web 优先断言。Playwright 断言是专门为动态网络创建的。检查会自动重试，直到满足必要的条件。
- 追踪。配置测试重试策略，捕获执行跟踪、视频、屏幕截图以定位问题。

运行机制

浏览器在不同进程中运行属于不同来源的 Web 内容。Playwright 与现代浏览器架构保持一致，并在进程外运行测试。这使得 Playwright 摆脱了典型的进程内测试运行器的限制。

- 多重一切。测试跨越多个选项卡、多个来源和多个用户的场景。为不同的用户创建具有不同上下文的场景，并在您的服务器上运行它们，所有这些都在一次测试中完成。
- 可信事件。悬停元素，与动态控件交互，产生可信事件。Playwright 使用与真实用户无法区分的真实浏览器输入管道。
- 测试框架，穿透 Shadow DOM。Playwright 选择器穿透影子 DOM 并允许无缝地输入帧。

完全隔离-快速执行

- 浏览器上下文。Playwright 为每个测试创建一个浏览器上下文。浏览器上下文相当于一个全新的浏览器配置文件。这提供了零开销的完全测试隔离。创建一个新的浏览器上下文只需要几毫秒。
- 登录一次。保存上下文的身份验证状态并在所有测试中重用它。这绕过了每个测试

中的重复登录操作，但提供了独立测试的完全隔离。

强大的工具

- 代码生成器。通过记录您的操作来生成测试。将它们保存为任何语言。
- 调试。检查页面、生成选择器、逐步执行测试、查看点击点、探索执行日志。
- 跟踪查看器。捕获所有信息以调查测试失败。Playwright 跟踪包含测试执行截屏、实时 DOM 快照、动作资源管理器、测试源等等。

1.3 playwright 环境准备

版本要求

python3.7+ 版本（课程推荐python3.8)版本

playwright 最新版 1.31.1

1.3.1 环境准备

Playwright 是专门为满足端到端测试的需要而创建的。Playwright 支持所有现代渲染引擎，包括 Chromium、WebKit（Safari 的浏览器引擎）和 Firefox。

在 Windows、Linux 和 macOS 上进行本地测试或在 CI 上进行测试。

python 版本要求 python3.7+ 版本。

安装 playwright :

```
pip install playwright
```

安装所需的浏览器 chromium,firefox 和 webkit :

```
playwright install
```

仅需这一步即可安装所需的浏览器，并且不需要安装驱动包了（解决了selenium启动浏览器，总是要找对应驱动包的痛点）

整个下载过程会比较慢，大概需要3-5分钟，它会下载3个内置浏览器，放到你电脑本地。

如果安装报错，提示缺少Visual C++，解决办法：

安装Microsoft Visual C++ Redistributable 2019

```
https://aka.ms/vs/16/release/VC_redist.x64.exe
```

直接点击就可以下载了，下载后直接安装即可。

1.3.2 运行代码快速体验

启动浏览器并打开百度页面

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False) # 启动 chromium 浏览器
    page = browser.new_page() # 打开一个标签页
    page.goto("https://www.baidu.com") # 打开百度地址
    print(page.title()) # 打印当前页面title
    browser.close() # 关闭浏览器对象
```

1.4 快速开始

1.4.1 简单使用

官方介绍<https://playwright.dev/python/>

安装后，您可以在 Python 脚本中使用 Playwright，并启动 3 种浏览器中的任何一种（chromium,firefox和webkit）。

启动浏览器并打开百度页面

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False) # 启动 chromium 浏览器
    page = browser.new_page() # 打开一个标签页
    page.goto("https://www.baidu.com") # 打开百度地址
    print(page.title()) # 打印当前页面title
    browser.close() # 关闭浏览器对象
```

Playwright 支持2种运行方式：同步和异步。如果您的现代项目使用asyncio，您应该使用 async API：

以下是异步运行方式

```
import asyncio
from playwright.async_api import async_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

async def main():
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=False)
        page = await browser.new_page()
        await page.goto("https://www.baidu.com")
        print(await page.title())
        await browser.close()

asyncio.run(main())
```

如果你不习惯with语句，也可以用start() 和stop() 的方式

```
from playwright.sync_api import sync_playwright

playwright = sync_playwright().start()

browser = playwright.chromium.launch(headless=False)
page = browser.new_page()
page.goto("https://www.baidu.com/")

browser.close()
playwright.stop()
```

1.4.2 headless 模式

默认情况下，Playwright 以无头模式运行浏览器。要查看浏览器 UI，请headless=False在启动浏览器时传递标志。

headless 无头模式运行浏览器示例：

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as p:
    browser = p.chromium.launch()           # 启动 chromium 浏览器
    page = browser.new_page()              # 打开一个标签页
    page.goto("https://www.baidu.com")     # 打开百度地址
    print(page.title())                    # 打印当前页面title
    browser.close()                        # 关闭浏览器对象
```

1.4.3 关于等待

Playwright 打开浏览器运行脚本的速度那就是一个字：快！

您还可以用来`slow_mo`（单位是毫秒）减慢执行速度。它的作用范围是全局的，从启动浏览器到操作元素每个动作都会有等待间隔，方便在出现问题的时候看到页面操作情况。

```
chromium.launch(headless=False, slow_mo=50)
```

使用示例

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False)
    page = browser.new_page()
    page.goto("https://www.baidu.com")
    print(page.title())
    page.fill('#kw', "上海-悠悠博客")
    page.click('#su')
    browser.close()
```

运行后会发现每个操作都会有间隔时间。

time.sleep() 不再使用

Playwright 在查找元素的时候具有自动等待功能，如果你在调试的时候需要使用等待，你应该使用`page.wait_for_timeout(5000)`代替`time.sleep(5)`并且最好不要等待超时。

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=1000)
    page = browser.new_page()
    page.goto("https://www.baidu.com")
    print(page.title())
    # 等待5秒
    page.wait_for_timeout(5000)
    page.fill('#kw', "上海-悠悠博客")
    page.click('#su')
    browser.close()
```

请使用 `wait( wait_for_timeout)` 方法而不是`time`模块。这是因为我们内部依赖于异步操作，并且在使用时`time.sleep(5)`无法正确处理它们。

playwright 可以支持自动录制生成脚本，也就是说只需要在页面上点点点，就可以自动生成对应的脚本了。

1.5 录制生成脚本

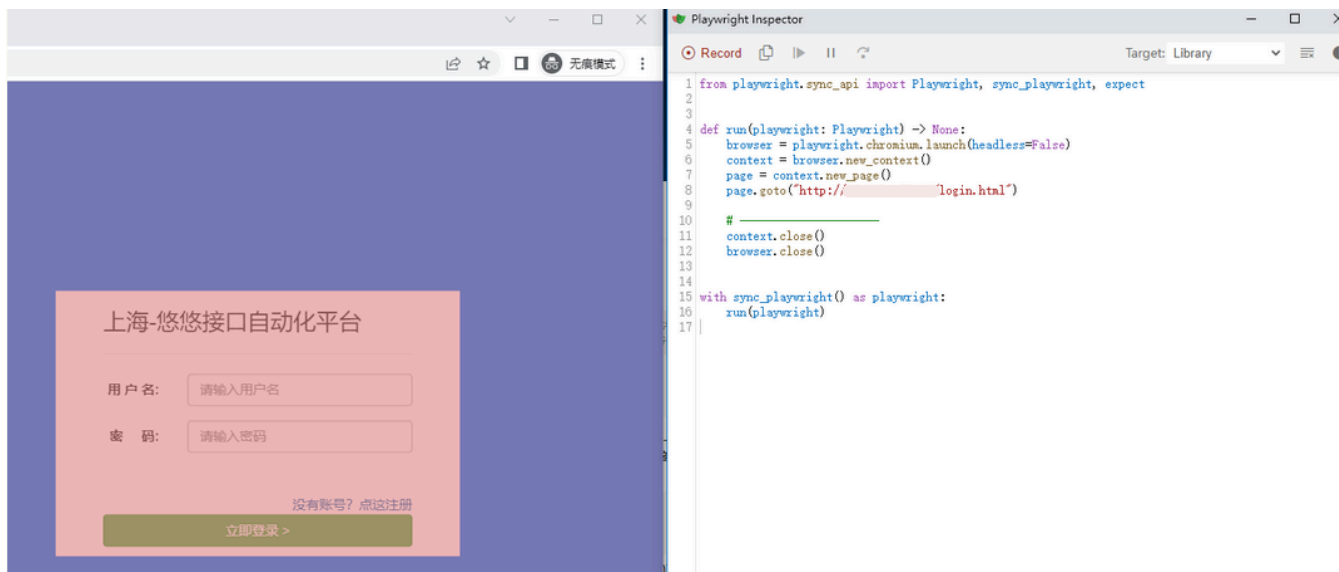
1.5.1 启动运行

Playwright 具有开箱即用的生成测试的能力，是快速开始测试的好方法。它将打开两个窗口，一个是浏览器窗口，您可以在其中与要测试的网站进行交互，另一个是 Playwright Inspector 窗口，您可以在其中记录测试、复制测试、清除测试以及更改测试语言。

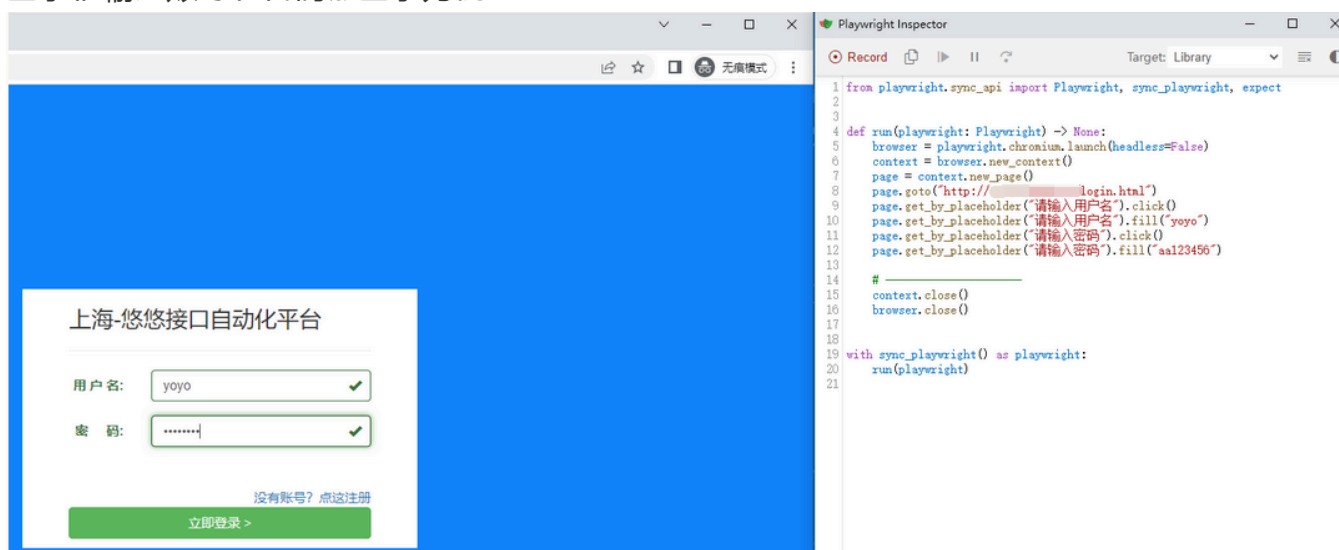
使用命令行启动

```
playwright codegen http://网站地址
```

启动后，电脑上出现2个窗口，左边是浏览器打开网站可以操作，右边是inspector 可以生成对应的脚本



登录框输入账号和密码点登录为例



一个完整的登录流程代码生成如下

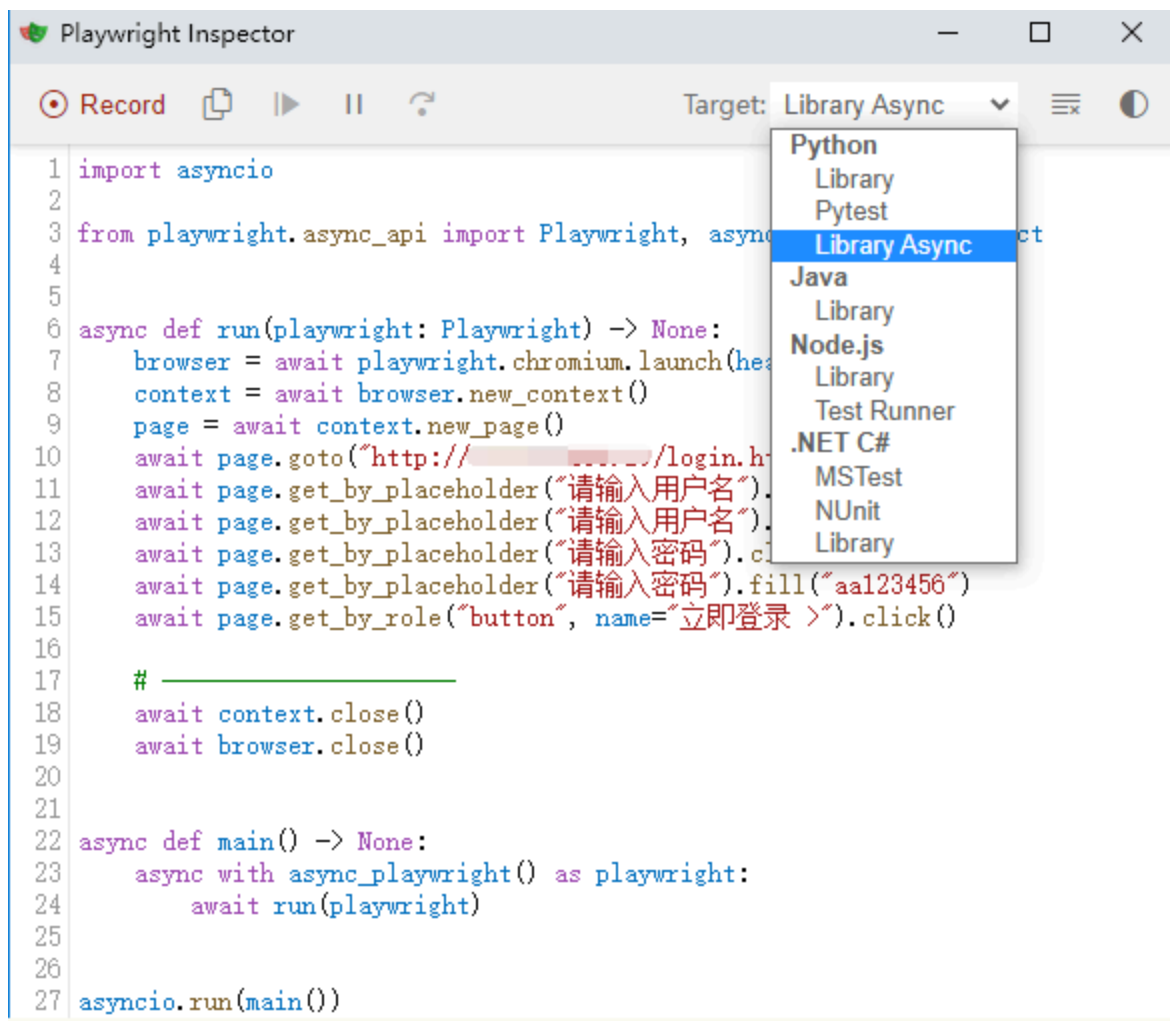
```
from playwright.sync_api import Playwright, sync_playwright, expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

def run(playwright: Playwright) -> None:
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    page.goto("http://127.0.0.1:8000/login.html")
    page.get_by_placeholder("请输入用户名").click()
    page.get_by_placeholder("请输入用户名").fill("yoyo")
    page.get_by_placeholder("请输入密码").click()
    page.get_by_placeholder("请输入密码").fill("aa123456")
    page.get_by_role("button", name="立即登录 >").click()

    # -----
    context.close()
    browser.close()

with sync_playwright() as playwright:
    run(playwright)
```

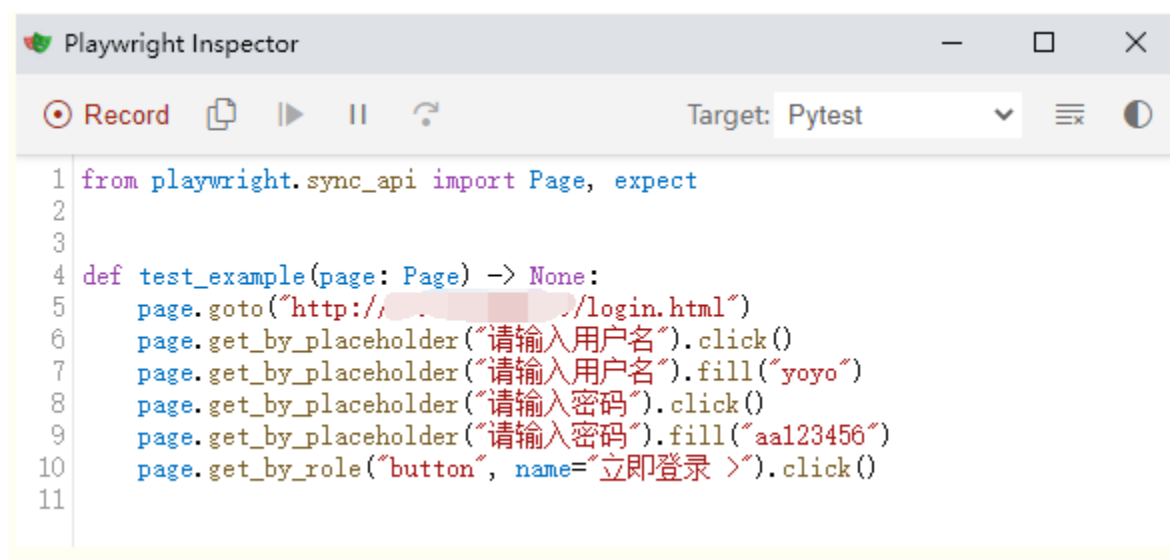
还可以选择生成异步代码



The image shows the Playwright Inspector window. The 'Target' dropdown is set to 'Library Async'. A dropdown menu is open, showing the following options: Python, Library, Pytest, Library Async (highlighted), Java, Library, Node.js, Library, Test Runner, .NET C#, MSTest, NUnit, and Library. The code in the background is a Python script using the 'asyncio' library and the 'playwright.async_api' module.

```
1 import asyncio
2
3 from playwright.async_api import Playwright, async_playwright
4
5
6 async def run(playwright: Playwright) -> None:
7     browser = await playwright.chromium.launch(headless=True)
8     context = await browser.new_context()
9     page = await context.new_page()
10    await page.goto("http://localhost:3000/login.html")
11    await page.get_by_placeholder("请输入用户名").click()
12    await page.get_by_placeholder("请输入用户名").fill("yoyo")
13    await page.get_by_placeholder("请输入密码").click()
14    await page.get_by_placeholder("请输入密码").fill("aa123456")
15    await page.get_by_role("button", name="立即登录 >").click()
16
17    # ...
18    await context.close()
19    await browser.close()
20
21
22 async def main() -> None:
23     async with async_playwright() as playwright:
24         await run(playwright)
25
26
27 asyncio.run(main())
```

如果你是写自动化测试用例，还能自动生成 pytest 框架的代码，简直了！



The image shows the Playwright Inspector window. The 'Target' dropdown is set to 'Pytest'. The code in the background is a Python script using the 'playwright.sync_api' module and the 'pytest' framework.

```
1 from playwright.sync_api import Page, expect
2
3
4 def test_example(page: Page) -> None:
5     page.goto("http://localhost:3000/login.html")
6     page.get_by_placeholder("请输入用户名").click()
7     page.get_by_placeholder("请输入用户名").fill("yoyo")
8     page.get_by_placeholder("请输入密码").click()
9     page.get_by_placeholder("请输入密码").fill("aa123456")
10    page.get_by_role("button", name="立即登录 >").click()
11
```

```
from playwright.sync_api import Page, expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

def test_example(page: Page) -> None:
    page.goto("http://127.0.0.1:8000/login.html")
    page.get_by_placeholder("请输入用户名").click()
    page.get_by_placeholder("请输入用户名").fill("yoyo")
    page.get_by_placeholder("请输入密码").click()
    page.get_by_placeholder("请输入密码").fill("aa123456")
    page.get_by_role("button", name="立即登录 >").click()
```

1.5.2 录制相关命令操作

相关命令参数：

1.codegen在浏览器中运行并执行操作

```
playwright codegen playwright.dev
```

2.Playwright 打开一个浏览器窗口，其视口设置为特定的宽度和高度，并且没有响应，因为需要在相同条件下运行测试。

使用该--viewport选项生成具有不同视口大小的测试。

```
playwright codegen --viewport-size=800,600 playwright.dev
```

3. --device 使用设置视口大小和用户代理等选项模拟移动设备时记录脚本和测试。

模拟移动设备iPhone11，注意：device的值必须用双引号，并且区分大小写

```
playwright codegen --device="iPhone 11" playwright.dev
```

4.模拟配色

```
playwright codegen --color-scheme=dark playwright.dev
```

5.模拟地理位置、语言和时区

```
playwright codegen --timezone="Europe/Rome" --geolocation="41.890221,12.492348" --lang="it-IT" maps.google.com
```

6.保留经过身份验证的状态

运行codegen以在会话结束时--save-storage保存cookie和localStorage。这对于单独记录身份验证步骤并在稍后的测试中重用它很有用。

执行身份验证并关闭浏览器后，auth.json将包含存储状态。

```
playwright codegen --save-storage=auth.json
```

运行--load-storage以消耗先前加载的存储。这样，所有的cookie和localStorage都将被恢复，使大多数网络应用程序进入身份验证状态。

```
playwright open --load-storage=auth.json my.web.app
playwright codegen --load-storage=auth.json my.web.app
# Perform actions in authenticated state.
```

1.5.3 page.pause() 断点调试

如果您想在某些非标准设置中使用codegen（例如，使用browser_context.route()），可以调用page.pause()，这将打开一个带有codegen控件的单独窗口。

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    # Make sure to run headed.
    browser = p.chromium.launch(headless=False)

    # Setup context however you like.
    context = browser.new_context() # Pass any options
    context.route('**/*', lambda route: route.continue_())

    # Pause the page, and start recording manually.
    page = context.new_page()
    page.pause()
```

1.6 pause断点 inspector 使用

在运行selenium脚本的时候，我们通常习惯用sleep去让页面暂停，打开console 输入

`$(selector)` 或者 `$x(xpath)` 去调试定位页面的元素。

有时候明明页面能找到元素，代码运行却找不到，很是郁闷！

playwright 的 `page.pause()` 断点功能出现, 让打开可以愉快的在页面上调试了, 我们甚至可以直接使用 `playwright.$(selector)` 直接支持playwright选择器的方法。

1.6.1 page.pause() 断点

在代码中加入 `page.pause()` 进入断点状态

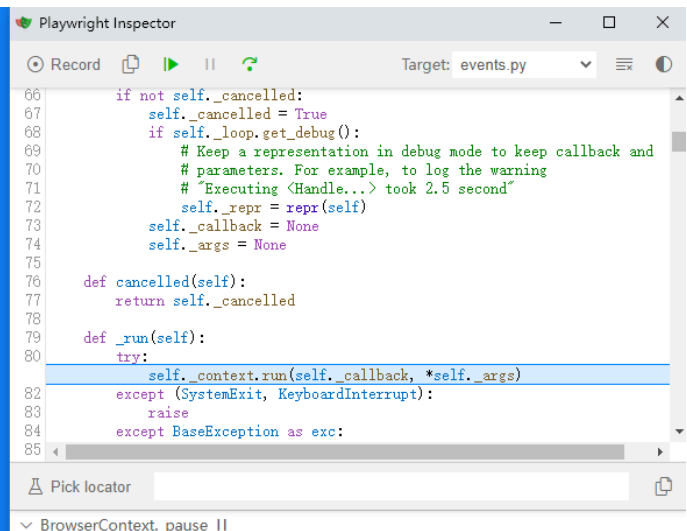
```
from playwright.sync_api import Playwright, sync_playwright, expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

def run(playwright: Playwright) -> None:
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    page.goto("http://127.0.0.1:8000/login.html")
    page.get_by_placeholder("请输入用户名").click()
    page.get_by_placeholder("请输入用户名").fill("yoyo")
    page.get_by_placeholder("请输入密码").click()
    page.get_by_placeholder("请输入密码").fill("aa123456")
    page.pause() # 断点
    page.get_by_role("button", name="立即登录 >").click()

    # -----
    context.close()
    browser.close()

with sync_playwright() as playwright:
    run(playwright)
```

运行后会弹出 `playwright inspector` 工具



1.6.2 console 调试定位

我们可以在 用代码打开的浏览器上f12 打开console页面，输入 `playwright.$(selector)` 调试定位



selector 语法可以支持 playwright 的selector 定位的语法。

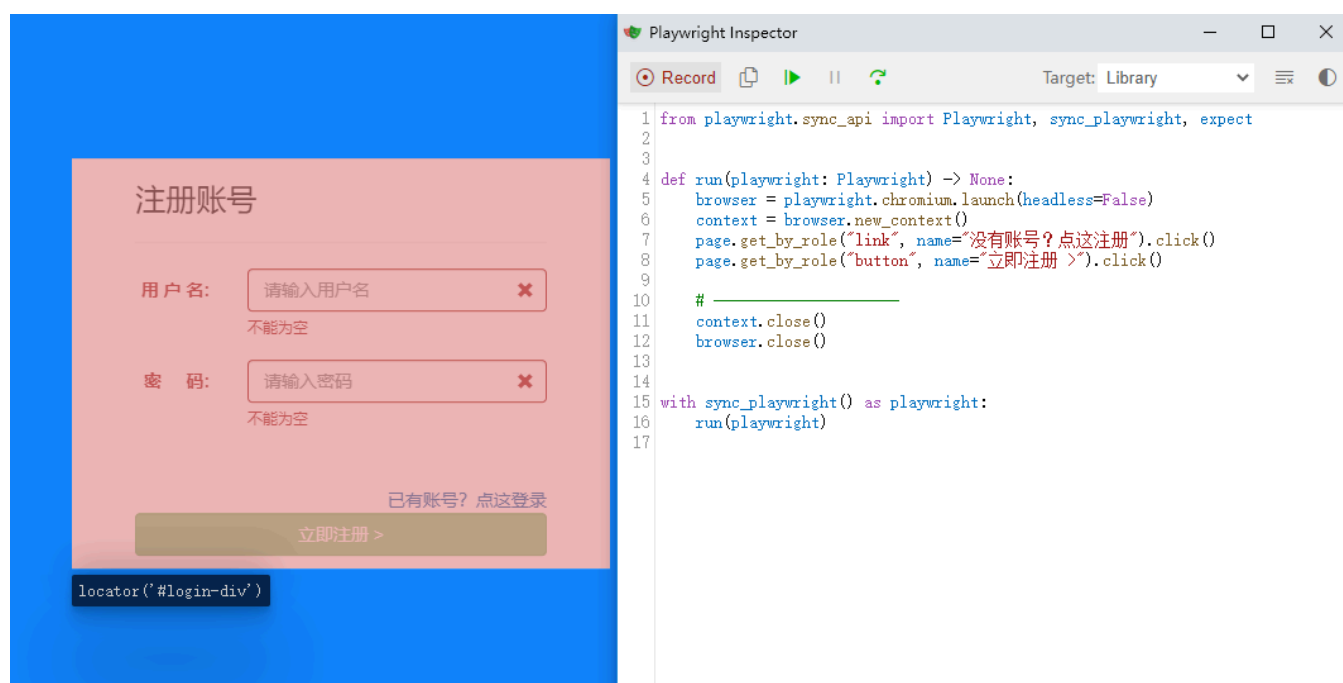
其它相关操作

- `playwright.$(selector)` 使用实际的Playwright查询引擎查询Playwright选择器

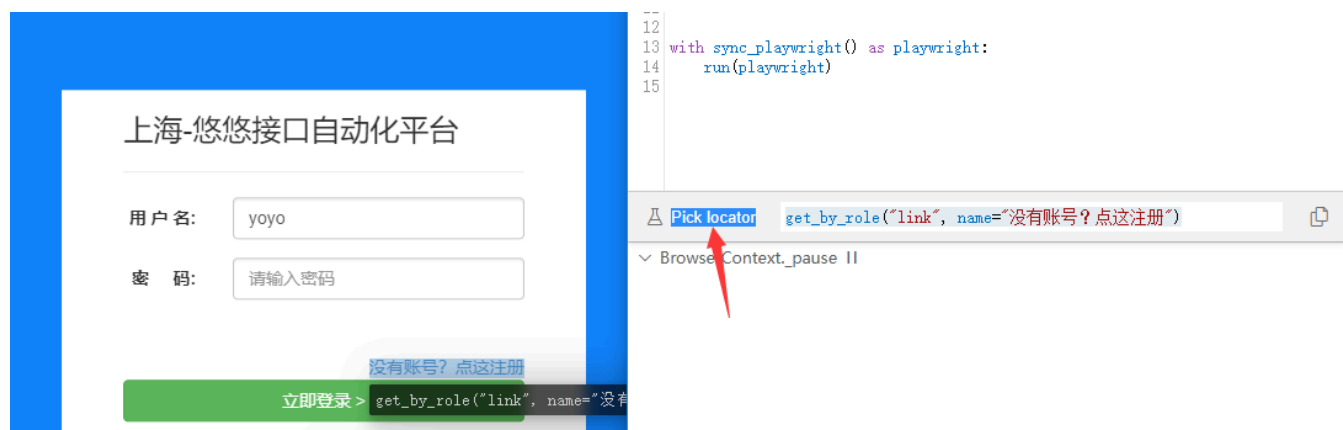
- `playwright.$$selector` 类似于 `playwright.$` ,但是返回全部的匹配元素
- `playwright.inspect(selector)` 在元素面板中显示元素（如果相应浏览器的 DevTools支持）。
- `playwright.locator(selector)` 使用实际的Playwright查询引擎查询Playwright元素
- `playwright.selector(element)` 为给定元素生成选择器。

1.6.3 playwright inspector 使用

还可以在playwright inspector 工具上点开启录制按钮，在页面上点点点，就可以生成对应的元素和操作



Pick locator 使用



点击 Pick locator 后在浏览器上选择需要定位的元素，即可生成对应的 locator

1.7 Playwright+Pytest+Allure POM项目

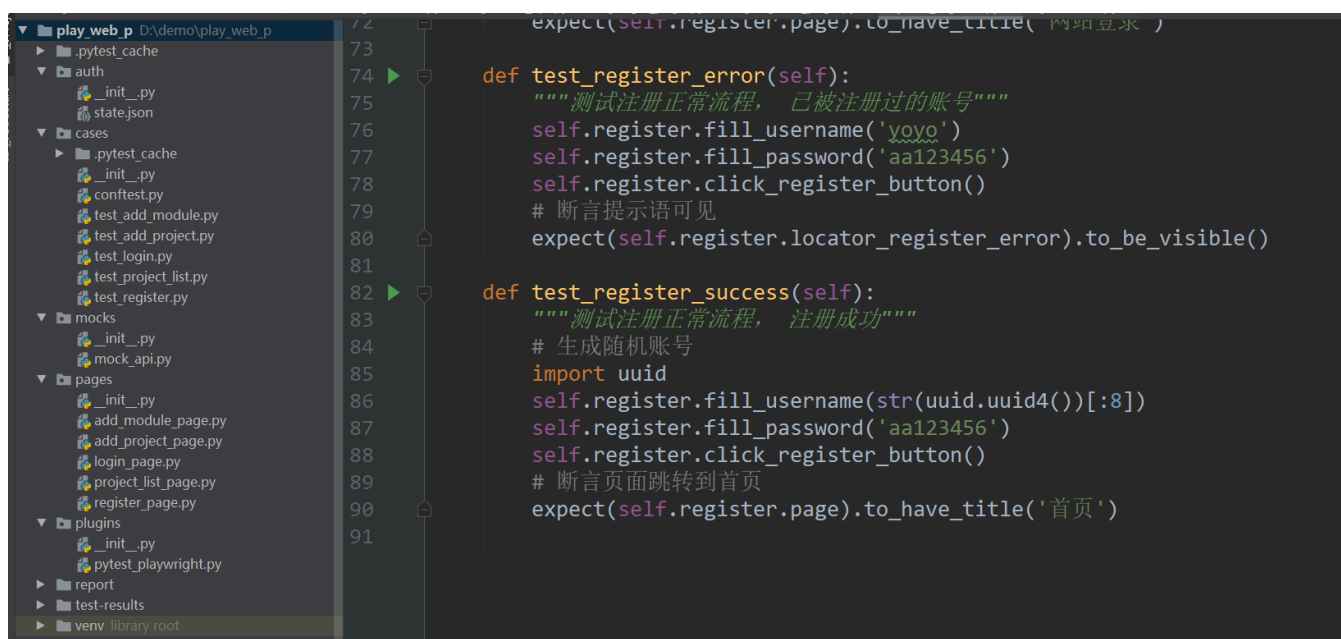
1.7.1 POM 设计模式

POM(Page Object Models) 页面对象模型已经成了写 web 自动化的一个标准模型。

页面对象代表 Web 应用程序的一部分。电子商务 Web 应用程序可能有一个主页、一个列表页面和一个结帐页面。它们中的每一个都可以由页面对象模型表示。

页面对象通过创建适合您的应用程序的更高级别的 API 来简化创作，并通过在一个地方捕获元素选择器和创建可重用代码来避免重复来简化维护。

采用传统的pom设计模型



```
72 expect(self.register.page).to_have_title('网站登录')
73
74 def test_register_error(self):
75     """测试注册正常流程， 已被注册过的账号"""
76     self.register.fill_username('yoyo')
77     self.register.fill_password('aa123456')
78     self.register.click_register_button()
79     # 断言提示语可见
80     expect(self.register.locator_register_error).to_be_visible()
81
82 def test_register_success(self):
83     """测试注册正常流程， 注册成功"""
84     # 生成随机账号
85     import uuid
86     self.register.fill_username(str(uuid.uuid4())[:8])
87     self.register.fill_password('aa123456')
88     self.register.click_register_button()
89     # 断言页面跳转到首页
90     expect(self.register.page).to_have_title('首页')
91
```

页面封装元素定位和操作方法

```

class RegisterPage:

    def __init__(self, page: Page):
        self.page = page
        self.locator_username = page.get_by_label("用户名:")
        self.locator_password = page.get_by_label("密码:")
        self.locator_register_btn = page.locator('text=立即注册')
        self.locator_login_link = page.locator('text=已有账号? 点这登录')
        # 用户名输入框提示语
        self.locator_username_tip1 = page.locator('[data-fv-validator="notEmpty"][data-')
        self.locator_username_tip2 = page.locator('[data-fv-validator="stringLength"][data-')
        self.locator_username_tip3 = page.locator('[data-fv-validator="regex"][data-')
        # 密码输入框提示语
        self.locator_password_tip1 = page.locator('[data-fv-validator="notEmpty"][data-')
        self.locator_password_tip2 = page.locator('[data-fv-validator="stringLength"][data-')
        self.locator_password_tip3 = page.locator('[data-fv-validator="regex"][data-')
        # 账号或密码不正确!
        self.locator_register_error = page.locator('text=用户名已存在或不合法! ')

```

```

def navigate(self):
    with allure.step("导航到注册页"):
        self.page.goto("/register.html")

def fill_username(self, username):
    with allure.step(f"输入用户名:{username}"):
        self.locator_username.fill(username)

def fill_password(self, password):
    with allure.step(f"输入密码:{password}"):
        self.locator_password.fill(password)

def click_register_button(self):
    with allure.step(f"点注册按钮"):
        self.locator_register_btn.click()

def click_login_link(self):
    with allure.step(f"点登录链接"):
        self.locator_login_link.click()

```

用例设计


```

2   from playwright.sync_api import expect
3   import pytest
4
5
6   class TestRegister:
7       """注册功能"""
8
9       @pytest.fixture(autouse=True)
10      def start_for_each(self, page):
11          print("for each--start: 打开新页面访问注册页")
12          self.register = RegisterPage(page)
13          self.register.navigate()
14          yield
15          print("for each--end: 后置操作")
16
17      def test_register_1(self):
18          """用户名为空, 点注册"""
19          self.register.fill_username('')
20          self.register.fill_password('123456')
21          self.register.click_register_button()
22          # 断言
23          expect(self.register.locator_username_tip1).to_be_visible()
24          expect(self.register.locator_username_tip1).to_contain_text("不能为空")

```

```

def test_register_error(self):
    """测试注册正常流程, 已被注册过的账号"""
    self.register.fill_username('yoyo')
    self.register.fill_password('aa123456')
    self.register.click_register_button()
    # 断言提示语可见
    expect(self.register.locator_register_error).to_be_visible()

def test_register_success(self):
    """测试注册正常流程, 注册成功"""
    # 生成随机账号
    import uuid
    self.register.fill_username(str(uuid.uuid4())[:8])
    self.register.fill_password('aa123456')
    self.register.click_register_button()
    # 断言页面跳转到首页
    expect(self.register.page).to_have_title('首页')

```

1.7.2 运行用例

使用pytest 命令行运行用例

```
>pytest --alluredir ./report

===== test session starts =====
platform win32 -- Python 3.8.5, pytest-7.2.1, pluggy-1.0.0
baseurl: http://47.108.155.10
rootdir: D:\demo\play_web_p, configfile: pytest.ini
plugins: allure-pytest-2.13.0, base-url-2.0.0
collected 41 items
```

查看报告

```
allure serve ./report
```

Allure

总览

类别

测试套

图表

时间刻度

功能

包

功能

order名称用时状态通过用例状态过滤: 104000Marks: 104000

> 注册功能10

> 添加模块4

> 添加项目7

✓ #2 异常场景-项目名称不能为空'chromium'247ms

✓ #1 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符", 'chromium', ", 'abcl@'171ms

✓ #6 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符", 'chromium', ", 'aaaaabbbbbbccccddddddeeee... 241ms

✓ #7 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符'aal@', 'chromium', ", 'abc'179ms

✓ #3 提交成功, 跳转到项目列表'chromium'532ms

✓ #5 服务器异常 500 状态码'chromium'305ms

✓ #4 项目名称重复, 弹出模态框'chromium'331ms

> 登录功能10

✗ #1 用户名为空, 点注册'chromium'186ms

✓ #2 用户名大于30字符'chromium'134ms

✓ #3 用户名有特殊字符'chromium'125ms

✓ #4 登录失败场景'chromium', '12345678', '输入错误的密码', 'yoyo'560ms

✓ #7 登录失败场景'chromium', '12345678', '输入错误的账号', 'yoyox1x2x3'580ms

✓ #6 登录正常流程, 登录成功'chromium'609ms

✓ #5 登录正常流程, 获取异步ajax 请求'chromium'413ms

> 项目列表12

✓ #1 用例描述0s

cases.test_add_project.TestAddProject#test_add_proj

通过 异常场景-项目名称, 无效等价: 特殊

总览 历史 重试次数

优先级: normal

耗时: 179ms

描述

异常场景-项目名称, 无效等价: 特殊字符/大于30个字符

参数

app: 'aal@'

browser_name: 'chromium'

desc: "

name: 'abc'

执行

> 前置

> 测试步骤

> 后置

用例失败截图和视频

order	名称	用时	状态	通过用例状态过滤: 1 0 40 0 0 0 Marks: [icon]
>	注册功能			10
>	添加模块			4
>	添加项目			7
✓	#2 异常场景-项目名称不能为空	'chromium'	247ms	
✓	#1 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符	", 'chromium', ", 'abcl@'	171ms	
✓	#6 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符	", 'chromium', ", 'aaaaabbbbbbccccdddeeee...	241ms	
✓	#7 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符	'aal@', 'chromium', ", 'abc'	179ms	
✓	#3 提交成功, 跳转到项目列表	'chromium'	532ms	
✓	#5 服务器异常 500 状态码	'chromium'	305ms	
✓	#4 项目名称重复, 弹出模态框	'chromium'	331ms	
>	登录功能			1 6
✗	#1 用户名为空, 点注册	'chromium'	186ms	
✓	#2 用户名大于30字符	'chromium'	134ms	
✓	#3 用户名有特殊字符	'chromium'	125ms	
✓	#4 登录失败场景	'chromium', '12345678', '输入错误的密码', 'yoyo'	560ms	
✓	#7 登录失败场景	'chromium', '12345678', '输入错误的账号', 'yoyox1x2x3'	580ms	
✓	#6 登录正常流程, 登录成功	'chromium'	609ms	
✓	#5 登录正常流程, 获取异步ajax 请求	'chromium'	413ms	
>	项目列表			12
✓	#1 用例描述		0s	

order	名称	用时	状态	通过用例状态过滤: 1 0 40 0 0 0 Marks: [icon]
>	注册功能			10
>	添加模块			4
>	添加项目			7
✓	#2 异常场景-项目名称不能为空	'chromium'	247ms	
✓	#1 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符	", 'chromium', ", 'abcl@'	171ms	
✓	#6 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符	", 'chromium', ", 'aaaaabbbbbbccccdddeeee...	241ms	
✓	#7 异常场景-项目名称, 无效等价: 特殊字符/大于30个字符	'aal@', 'chromium', ", 'abc'	179ms	
✓	#3 提交成功, 跳转到项目列表	'chromium'	532ms	
✓	#5 服务器异常 500 状态码	'chromium'	305ms	
✓	#4 项目名称重复, 弹出模态框	'chromium'	331ms	
>	登录功能			1 6
✗	#1 用户名为空, 点注册	'chromium'	186ms	
✓	#2 用户名大于30字符	'chromium'	134ms	
✓	#3 用户名有特殊字符	'chromium'	125ms	
✓	#4 登录失败场景	'chromium', '12345678', '输入错误的密码', 'yoyo'	560ms	
✓	#7 登录失败场景	'chromium', '12345678', '输入错误的账号', 'yoyox1x2x3'	580ms	
✓	#6 登录正常流程, 登录成功	'chromium'	609ms	
✓	#5 登录正常流程, 获取异步ajax 请求	'chromium'	413ms	
>	项目列表			12
✓	#1 用例描述		0s	

stdout

159 B

AssertionError: assert 1 == 2

后置

page::0

0s

context::0 2个附件

525ms

test_login_1[chromium]-failed-1

20.6 KB

test_login_1[chromium]-failed-1

22.9 KB

stdout

159 B

AssertionError: assert 1 == 2

后置

page::0

0s

context::0 2个附件

525ms

test_login_1[chromium]-failed-1

20.6 KB

test_login_1[chromium]-failed-1

22.9 KB

playwright::0

34ms

start_for_each::0

1ms

browser::0

40ms

激活 Windows

Go to Settings to activate Windows.

网易云视频课程

网易云视频完整课程地址<https://study.163.com/course/courseMain.htm?courseId=1213382811&share=2&shareId=480000002230338>

Playwright + Python

Web 自动化测试

上海-悠悠